MERGE-SPLIT DYNAMIC TOKENIZATION FOR FLEXIBILITY

Justin Chae, Hanbyul Kang, Alvin Zeng
Paul G. Allen School of Computer Science & Engineering
University of Washington
{jchae3, kahaby, wnzeng}@cs.washington.edu

ABSTRACT

Dynamic tokenization adapts token boundaries to the input at inference time, shortening sequences without retraining the underlying model. Existing approaches are *merge-only*: they coarsen the tokenization but cannot restore finer granularity when aggressive merging collapses linguistically meaningful boundaries—a particular risk for morphologically complex languages. We introduce a two-stage, inference-time *merge–split* pipeline that augments dynamic BPE and cross-word SuperBPE merging with an entropy-guided split stage: tokens whose next-token prediction entropy exceeds a threshold τ are re-tokenized into finer units. Using a frozen Mistral-7B with a pretrained embedding hypernetwork, we evaluate each stage in isolation and in combination on English (MMLU) and Korean (KMMLU, KoBEST), and extend the study to six further models spanning a range of Korean training data. Our analysis paints a nuanced picture: neither merging nor splitting improves over the unmodified model, and the hypernetwork's embedding approximation—not segmentation itself—is the dominant cost, especially for Korean. Splitting is nonetheless a useful *corrective*: in the aggressive cross-word merging regime where merge-only accuracy collapses toward chance, entropy-guided splitting recovers up to 19 points on MMLU. We characterize when dynamic tokenization helps in terms of token-distribution shift, entropy-threshold sensitivity, and base-tokenizer granularity. [1]

1 INTRODUCTION

Dynamic tokenization has recently been proposed as a way to improve inference efficiency by adapting token boundaries to input at runtime. Feher et al. (2025) demonstrate that dynamically merging frequent subword sequences reduces sequence length while maintaining competitive performance, using a pretrained hypernetwork (Minixhofer et al., 2024) to generate embeddings for newly formed tokens without retraining the base model. However, existing dynamic tokenization approaches are largely *merge-only*: they allow tokens to be combined into coarser units but provide no mechanism for splitting over-merged tokens when finer granularity is needed. The desirable level of granularity is context-dependent—particularly for morphologically complex languages, where aggressive merging can collapse boundaries that carry important semantic or grammatical information. At the same time, Cognetta et al. (2024) show that minimizing token count does not always improve model performance; in some settings, finer-grained tokenization better preserves semantic fidelity. Together, these observations suggest that a purely merge-based strategy is inherently limited and that adding a *split* operation would offer greater flexibility.

This work explores dynamic tokenization from within-word level token merging, super-word token merging, to token splitting. We design and evaluate a two-stage inference-time pipeline that combines Dynamic BPE merging with entropy-guided splitting, while also ablating each dynamic tokenization method to rigorously investigate when dynamic tokenization helps. The pipeline first applies BPE merges guided by within batch co-occurrence statistics, then scans the prefill entropy of each token and re-tokenizes those exceeding a threshold τ—decomposing high-uncertainty tokens

[1] Our code and evaluation harness are available at https://github.com/HuNtErJ1324/dynamic-tokenization.

into finer sub-units. We focus on Korean as a primary test case, since it represents a challenging setting where English-centric tokenizers fall back to byte-level representations and the tension between merging and splitting is most acute.

In practice, our results reveal a more complex picture than the initial motivation suggests. Splitting alone hurts English MMLU performance with accuracy dropping from 60.25% to 51.44%, and dynamic merging alone also hurts Korean KMMLU with accuracy dropping from 36.39% to 19.18%. Our hypothesis is that altering token boundary distributions away from the pretrained token embedding distribution degrades performance. This leads us to ask not just whether merge-split is better than merge-only, but under what conditions dynamic tokenization helps at all. To investigate, we extend evaluation to KoBEST (Jang et al., 2022) and KMMLU (Son et al., 2024), and broaden the analysis to six additional models with varying amounts of Korean training data. Our main contributions are:

- A two-stage merge-split pipeline combining Dynamic BPE merging and entropy-guided splitting, operating entirely at inference time.
- An entropy-guided splitting criterion with language-specific heuristics for Korean morphology.
- Analysis of token distribution shift, entropy threshold sensitivity, and Korean training data as factors governing dynamic tokenization effectiveness.

2 Related Work

2.1 Subword Tokenization

Byte-Pair Encoding (Sennrich et al., 2016) and SentencePiece (Kudo & Richardson, 2018) have become the dominant tokenization approaches for large language models. Both methods learn a fixed vocabulary from a training corpus through iterative subword merging, producing a static tokenizer that is applied uniformly at inference. While effective for languages well-represented in the training data, these approaches can produce over-fragmented representations for morphologically rich or low-resource languages (Park et al., 2020). Cognetta et al. (2024) provide empirical evidence that minimizing token count—the primary objective of BPE—does not always improve downstream performance, and that finer-grained tokenization can be beneficial in some settings.

2.2 Korean NLP and Multilingual Benchmarks

Korean presents distinctive challenges for tokenization due to its agglutinative morphology and the mismatch between Hangul syllable blocks and BPE vocabularies trained on English-dominant corpora. Park et al. (2020) show that the optimal tokenization strategy for Korean NLP tasks is task-dependent, and that morphological segmentation combined with BPE can outperform plain BPE on several downstream tasks.

To evaluate our approach on Korean, we use KMMLU (Son et al., 2024), a Korean counterpart of MMLU covering 45 subjects, and KoBEST (Jang et al., 2022), a benchmark of five Korean downstream tasks designed by professional linguists to test knowledge of Korean linguistic structure.

3 Background

3.1 Byte-Pair Encoding

A tokenizer is a map $T : \Sigma^* \to V^*$ that segments a string x over an alphabet Σ into tokens $T(x) = (t_1, \ldots, t_n)$ from a fixed vocabulary V. We write $|T(x)| = n$ for the sequence length. A model f_θ with parameters θ reads these tokens through an input embedding matrix $E \in \mathbb{R}^{|V| \times d}$, retrieving the row $E_t \in \mathbb{R}^d$ for each token t. Byte-pair encoding (Sennrich et al., 2016) fixes both V and T before training: from a base alphabet V_0 it learns an *ordered* list of merge rules $\mu = (\mu_1, \ldots, \mu_k)$, each greedily joining the most frequent adjacent pair in the corpus,

$$\mu_i : (a, b) \mapsto ab, \qquad (a, b) = \underset{(a', b')}{\arg\max}\ \mathrm{count}(a', b'), \tag{1}$$

so that $V = V_0 \cup \{ab : (a, b) \in \mu\}$. At inference μ is frozen and applied deterministically; T never adapts to the input, domain, or language at hand.

This rigidity falls unevenly across languages. A vocabulary fit to an English-dominated corpus oversegments other scripts and morphologies, so for parallel strings x_{en}, x_ℓ of equal meaning one typically observes $|T(x_\ell)| \gg |T(x_{\text{en}})|$ (Rust et al., 2021; Petrov et al., 2023). Because compute, latency, and API cost all scale with token count, this is a fairness problem as much as an efficiency one (Petrov et al., 2023). Korean is a representative case: its agglutinative morphology and Hangul script are heavily fragmented by English-centric BPE, inflating $|T(x)|$ for content identical to English (Section 6.3).

3.2 Dynamic Tokenization

Feher et al. (2025) keep f_θ frozen but let T depend on the input. For a batch $B = \{x^{(1)}, \ldots, x^{(b)}\}$ they run BPE *on the batch itself* under a budget of m merges, yielding a batch-specific merge list μ_B and an augmented vocabulary

$$V_B = V_0 \cup M_B, \qquad M_B = \{ab : (a, b) \in \mu_B\}, \qquad |M_B| \le m, \tag{2}$$

where V_0 is now the model's *original* subword vocabulary. Re-tokenizing with the induced T_B never lengthens a sequence, $|T_B(x)| \le |T_0(x)|$, and the budget m trades length against vocabulary novelty. The catch is that the new tokens $t \in M_B$ have no row in E; dynamic tokenization supplies them with a hypernetwork $\mathcal{H}_\phi$ (Minixhofer et al., 2024), so the model reads the merged sequence through the effective embedding

$$\widetilde{E}_t = \begin{cases} E_t & t \in V_0, \\ \mathcal{H}_\phi(t) & t \in M_B, \end{cases} \tag{3}$$

reusing pretrained rows and predicting only the merged tokens. With per-batch compression $\rho = 1 - \frac{1}{|B|} \sum_{x \in B} |T_B(x)|/|T_0(x)|$, Feher et al. (2025) report $\rho > 20\%$ across 14 languages on the encoder XLM-R at under 2% degradation, and up to $\rho \approx 17\%$ on the decoder Mistral-7B with minimal degradation. For decoders the savings apply to the prefill and scoring passes, where the full input is available to re-tokenize, rather than to autoregressive generation. Our experiments reuse this setting—a frozen Mistral-7B and the pretrained hypernetwork at inference time (Section 4).

3.3 SuperBPE: Cross-Word Merging

Standard BPE is constrained by *pretokenization*: a regular-expression step first splits a string at whitespace into pretokens $w_1, \ldots, w_p$, and the merge rules of Eq. equation 1 are applied independently within each w_i. No token may span a whitespace boundary, which bounds how far a vocabulary of fixed size can compress text—frequent multi-word units such as "of the" can never collapse into a single token. SuperBPE (Liu et al., 2025) lifts this constraint with a two-phase merge curriculum governed by a transition point $\kappa \in \{0, \ldots, k\}$:

$$\underbrace{\mu_1, \ldots, \mu_\kappa}_{\text{subwords: merge within a pretoken}} \;\Big|\; \underbrace{\mu_{\kappa+1}, \ldots, \mu_k}_{\text{superwords: merge across whitespace}} . \tag{4}$$

The first κ merges respect the within-pretoken constraint and learn ordinary subwords; the remainder drop it, so adjacent pretokens may be joined into multi-word *superword* tokens. The transition point interpolates between standard BPE ($\kappa = k$, no superwords) and fully unconstrained merging ($\kappa = 0$). At a matched vocabulary size of 200k, SuperBPE encodes text in up to a third fewer tokens than BPE while *improving* downstream accuracy (e.g. an 8.2-point gain on MMLU); and because new superwords keep surfacing common phrases, it continues to benefit from larger vocabularies long after BPE saturates (Liu et al., 2025). As a tokenizer-only change it composes directly with the per-batch hypernetwork setting of Section 3.2: cross-word merges simply add more out-of-vocabulary tokens for $\mathcal{H}_\phi$ to embed. Our merge stage adopts this curriculum with κ as a tunable knob (Section 4).

4 METHOD

4.1 ENTROPY-GUIDED SPLIT

We introduce an adaptive tokenization method driven by autoregressive entropy. Given an input text sequence X, an autoregressive language model M, and a base tokenizer T, we first apply T to X to generate an initial token sequence $B = (b_1, b_2, \ldots, b_n)$. During the prefill phase, M computes the next-token probability distributions across B, producing Shannon entropies $H = (H_1, H_2, \ldots, H_n)$. Our entropy-based splitting algorithm identifies target tokens b_i where the local entropy H_i exceeds a predefined threshold τ:

$$\mathcal{S} = \{b_i \in B \mid H_i > \tau\} \tag{5}$$

All such high-entropy tokens $b_i \in \mathcal{S}$ are subsequently re-tokenized into finer sub-token sequences, transforming B into a more morphologically aware sequence B' prior to downstream inference.

To execute this re-tokenization, the algorithm inserts a tentative segmentation boundary within the underlying string represented by the target token b_i, partitioning it into two segments that are separately re-tokenized by T. The algorithm iteratively shifts the boundary to identify the split point that minimizes the number of newly generated tokens. By minimizing the expansion of a single token, this approach extracts valid morphemes from the underlying text and preserves the structural integrity of the token sequence. To prevent unnecessary splitting, we implement language-specific heuristics:

- **English:** Words that are shorter than a predefined length are excluded, since they typically consist of a single morpheme (such as function words).
- **Korean:** The algorithm allows individual character blocks to be decomposed into their constituent sub-syllabic components (Jamos).

The rationale for using autoregressive entropy is that an elevated H_i signifies model uncertainty about the local context given the previous sequence $(b_1, b_2, \ldots, b_i)$. Decomposing the high-entropy token b_i exposes its morphological features, helping the model process the semantic content of the input text and predict with more certainty. We hypothesize that this adaptive adjustment improves context comprehension and boosts model performance across different benchmarks—especially within morphologically rich languages such as Korean.

4.2 MERGE–SPLIT PIPELINE

Our full method composes the two operations into a single inference-time pass over an input X. **Stage 1 (merge)** applies either within-word dynamic BPE (Eq. equation 2) or cross-word SuperBPE (Eq. equation 4) to the batch, shortening the sequence and embedding the newly merged tokens with the hypernetwork $\mathcal{H}_\phi$. **Stage 2 (split)** takes the resulting token sequence, runs the prefill pass to obtain the per-token entropies H_i, and re-tokenizes every token in $\mathcal{S} = \{b_i : H_i > \tau\}$ into finer units with the original tokenizer, producing the final sequence B' that is scored. The two stages pull in opposite directions—merging shortens the sequence while splitting lengthens it—so the merge budget m (or transition point κ) and the entropy threshold τ jointly determine where the sequence lands on the compression–fidelity axis. The limiting cases recover the ablations of Section 6: $\tau \to \infty$ gives merge-only dynamic tokenization, $m = 0$ gives split-only, and $m = 0,\ \tau \to \infty$ is the plain model. Both stages are training-free and operate entirely on the frozen model.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

We employ the Mistral-7B architecture as our primary model for evaluation. To support vocabulary merging in dynamic BPE and SuperBPE, we employ a hypernetwork (Minixhofer et al., 2024) specialized for predicting token embeddings for Mistral-7B. Our choice of Mistral-7B is motivated by the availability of a pre-trained hypernetwork for the model. For adaptive token splitting, we use the model's original tokenizer to perform all necessary re-tokenization. Every configuration is run at

inference time on the frozen model, with no fine-tuning; the model weights θ and the hypernetwork $\mathcal{H}_\phi$ are held fixed throughout.

We compare six configurations: two baselines that vary the embeddings or segmentation in isolation, and four dynamic-tokenization methods that constitute our contribution.

Baselines.

- **Plain.** The original tokenizer with the model's pretrained embeddings, i.e. the unmodified base model.
- **Original tokenizer + hypernetwork.** The original tokenization, but each token embedding is replaced by the hypernetwork prediction $\mathcal{H}_\phi(t)$. This isolates the approximation error of $\mathcal{H}_\phi$ from any change in segmentation.

Dynamic tokenization.

- **Within-word merging (dynamic BPE).** Per-batch BPE merges confined to pretoken boundaries equation 2, with $\mathcal{H}_\phi$ supplying embeddings for the merged tokens.
- **Super-word merging (SuperBPE).** The same per-batch merging under the cross-word schedule equation 4, governed by the transition point κ, so that merges may span whitespace.
- **Entropy-guided splitting.** The original tokenization, after which every token whose predictive entropy exceeds the threshold τ is re-tokenized into finer units by the original tokenizer.
- **Merge–split (ours).** Dynamic BPE merging followed by entropy-guided splitting of the high-entropy merged tokens—the full pipeline that trades compression against fidelity.

5.2 English Benchmark: MMLU

We evaluate the performance of entropy-based splitting and dynamic merging approaches on the English Massive Multitask Language Understanding (MMLU) benchmark (Hendrycks et al., 2021), which covers over 10K multiple-choice questions from 57 subjects to assess the model's reasoning and language comprehension capabilities. For the adaptive splitting approach, we analyze the overall accuracy across a set of different threshold values ranging from 1.0 to 10.0. Increasing the threshold decreases the likelihood of splitting. A threshold of 10.0 closely approaches the maximum entropy for a 32K vocabulary size language model. For the merge strategy, we evaluate superBPE across varying transition points, where lower transition points lead to earlier cross-word merges.

In addition to analyzing the overall accuracies, we examine per-subject performances and conduct qualitative case analyses on representative examples to better demonstrate how splitting affects the model inference.

5.3 Korean Benchmarks: KMMLU, KoBEST

We evaluate on two Korean benchmarks that test complementary aspects of language understanding.

KMMLU (Son et al., 2024) is the Korean counterpart of MMLU, covering 45 subjects spanning science, law, engineering, and the humanities. Each subject contains 100 test examples in a 4-choice multiple-choice format. We use all 45 subjects (35,030 examples total) and report average accuracy. We use 5-shot prompting with examples drawn from the per-subject development sets, and score each choice by computing the log-likelihood of the choice text appended to the prompt.

KoBEST (Jang et al., 2022) is a Korean benchmark designed by professional linguists, consisting of five tasks that require Korean-specific linguistic knowledge: BoolQ (yes/no reading comprehension), COPA (causal commonsense reasoning), WiC (word sense disambiguation), HellaSwag (sentence completion), and SentiNeg (sentiment analysis). We evaluate on the full test splits (397–1,404 examples per task) using 5-shot prompting from the validation set. For all tasks, we score by comparing the log-likelihood of each candidate choice text.

Table 1: Baseline accuracy. Replacing the pretrained embeddings with hypernetwork predictions —with *no* change in segmentation (*original tokenizer + hypernetwork*)—already costs accuracy, severely so for Korean, establishing a floor that every merge-based method inherits.

Setting	MMLU (en) %	KMMLU (ko) %
Plain (frozen Mistral-7B)	60.2	36.3
Original tokenizer + hypernetwork	57.0	21.9
Entropy split only ($\tau = 3$)	55.5	36.0

Table 2: Cross-word (SuperBPE) merging *without* and *with* entropy splitting ($\tau = 3$) as the merge-stage compression target increases. Splitting recovers accuracy lost to aggressive merging, with the largest gains exactly where merge-only collapses toward the 25% chance level.

	MMLU (en) acc. %		KMMLU (ko) acc. %	
Merge reduction (%)	Merge	Merge–split	Merge	Merge–split
10	46.7	46.4	25.6	26.4
20	28.3	37.4	27.0	30.0
30	25.5	41.2	27.3	31.1
40	26.3	45.2	24.0	27.5
50	25.1	44.3	18.7	26.9

For KMMLU and KoBEST, we evaluate all seven models under both `plain` and `entropy_split` ($\tau = 3.0$) conditions. For Mistral-7B, which has a compatible hypernetwork, we additionally evaluate `dynamic_bpe` and `dynamic_bpe_entropy_split`.

6 Results and Analysis

6.1 Merge vs. Split vs. Merge-Split

We compare the three operations of our pipeline—merging only (dynamic BPE), splitting only (entropy split), and the combined merge–split—against the plain and hypernetwork baselines on MMLU (en) and KMMLU (ko); all accuracies are computed on a fixed evaluation subsample. The central result is that **no dynamic configuration exceeds the plain baseline** on either task (Table 1). The methods differ instead in how gracefully they trade accuracy for sequence compression, and splitting is best understood as a *corrective* on aggressive merging rather than a standalone gain (Table 2).

Baselines Plain Mistral-7B is the strongest configuration (60.2% MMLU, 36.3% KMMLU). Merely swapping the pretrained embeddings for hypernetwork predictions on the *unchanged* tokenization costs 3.3 points on MMLU but a striking 14.4 points on KMMLU—to 21.9%, below the 25% four choice multiple choice chance level. The embedding-prediction error of $\mathcal{H}_\phi$, not segmentation, is thus the dominant source of degradation for Korean, and it lower-bounds every method that embeds merged tokens through the hypernetwork.

Splitting alone never helps. Entropy splitting performs no compression (reduction $= 0$) and never improves on plain. On MMLU it degrades monotonically as the threshold falls, from essentially the plain level at high τ to 51.7% at $\tau = 1$ (see the entropy-threshold analysis below). On KMMLU it is essentially a no-op: Mistral already segments Korean into syllable- and byte-level units, so there is little to split (split rate ≈ 0) and accuracy stays at the plain level (36.0%). Splitting also adds an entropy-scan forward pass, roughly doubling latency. As a standalone operation it is therefore strictly dominated.

Merging alone trades accuracy for compression. Within-word merging (pretoken boundary) is the gentlest setting: at $\approx 14\%$ reduction it costs 6.1 points on MMLU ($60.2 \rightarrow 54.1\%$), while on KMMLU the merges rarely fire—Korean is already near byte-level, triggering early termination—so accuracy sits at the hypernetwork floor ($\approx 23\%$). Cross-word merging (superBPE) reaches far

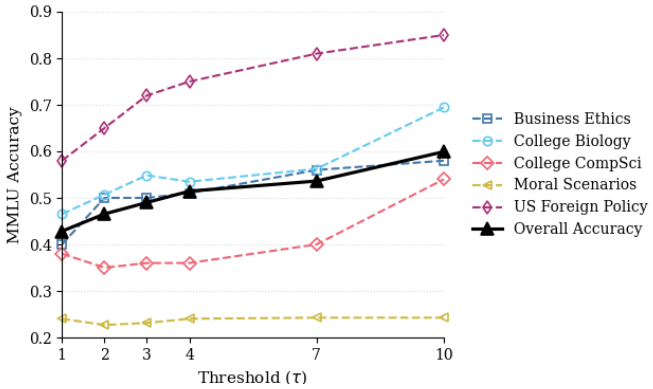

Figure 1: MMLU Subject Accuracies across Different Entropy Thresholds

higher compression (30–50%) but is destructive: MMLU collapses to the 25% chance level and KMMLU to 19–27% (Table 2, "Merge" columns). The merged superwords are out-of-distribution for the frozen model and only approximately embedded by $\mathcal{H}_\phi$, so beyond a modest budget, merging alone destroys the task signal.

Merge–split rescues the aggressive regime. The value of the combination appears precisely where merging fails. At matched cross-word compression, adding entropy splitting ($\tau = 3$) lifts MMLU by up to $+18.9$ points ($26.3 \rightarrow 45.2\%$ at 40% reduction) and KMMLU by up to $+8.2$ points ($18.7 \rightarrow 26.9\%$ at 50% reduction), and the gain *grows* with merge aggressiveness (Table 2); at the gentlest setting (10%) the split is negligible (rate $\rightarrow 0$, merge–split $\approx$ merge). This is the mechanism implied by our threshold analysis: aggressive merging manufactures exactly the high-entropy, out-of-distribution superwords that splitting targets ($H_j > \tau$), and re-fragmenting them restores units the hypernetwork can embed. Splitting thus repairs over-merging; it is synergistic with merging but not a source of gain on its own. The effect is asymmetric across languages: for English, splitting helps only in the cross-word regime (within-word merge–split, 52.0%, slightly trails merge-only, 54.1%), whereas for Korean it adds 3–5 points across *all* merge settings, since even moderate Korean merges yield poorly-embedded superwords.

Costs and caveats. Three caveats temper the recovery. (i) *Net compression.* The reduction in Table 2 is the *merge-stage* figure; splitting re-expands the sequence (split rate up to 0.55 at MMLU, 50%), so the realized compression of merge–split is substantially smaller—the accuracy is bought back partly with the merge's own savings. (ii) *Latency.* The split adds an entropy forward pass and re-embedding which greatly increases the latency. (iii) *Is entropy the right signal?* Replacing entropy-targeted splits with random splits of the same count costs only ≈ 1 point (MMLU 52.0 vs. 50.7%; KMMLU ties to $+1.1$), so entropy is at best a weakly informative criterion.

In summary, at a fixed compression budget the ordering is plain $>$ merge–split $>$ merge in the aggressive regime, with split-only offering neither compression nor an accuracy benefit. Merge–split does not raise raw accuracy, but it makes high-ratio compression far less destructive—the regime in which a dynamic tokenizer would actually be deployed.

6.2 Entropy Threshold Sensitivity

As illustrated in Figure 1, the empirical performance of adaptive splitting on MMLU benchmark indicates a steady decline in performance as token splitting becomes more aggressive. A closer inspection at the per-subject accuracies reveals significant performance degradation in two intervals of τ:

- $\tau \in [7.0, 10.0]$: An initial performance drop although only very few tokens are re-tokenized.
- $\tau \in [1.0, 3.0]$: An interval where token split becomes more aggressive and frequent.

We also observe two exceptions to this trend. First, performance on the college computer science subject has a slight recovery in accuracy when $\tau = 1.0$ after demonstrating a constant decrease from $\tau = 10.0$ to $\tau = 2.0$. Second, performance on the moral scenario subject remains low and consistent over all threshold values with a slight improvement at $\tau = 1.0$.

We hypothesize two main factors for this drop in accuracy. First, splitting creates a mismatch between the token distribution during inference time and the token distribution during pretraining. The model's BPE tokenizer always produces deterministic token sequences for training and inference. Therefore, an arbitrary adjustment to the token boundaries will produce a suboptimal, fragmented token sequence for the downstream task, degrading the model's comprehension of the input text. The drastic drop in accuracy of college biology accuracy for $\tau \in [7.0, 10.0]$ supports this hypothesis, as the model exhibits high sensitivity to even little shifts in token distribution. In addition, the positional encoding of the tokens is also altered by token splitting. The sequence expansion stretches the mathematical distance among tokens. As a result, two subword tokens can be unexpectedly far from each other, making it difficult for the model to group subword tokens and produce optimal attention values.

To explore further, we closely examine how our split approach works on specific MMLU prompts and the corresponding model predictions. We observe a systematic performance cliff: predictions generated at thresholds above $\tau = 3.0$ yield correct answers, while predictions generated at threshold below $\tau = 3.0$ produces incorrect ones. This observation aligns with the sharp accuracy drop we found in the interval from $\tau = 3.0$ to $\tau = 1.0$.

We attribute this performance cliff to the decomposition of important context keywords under aggressive split setting. Given a MMLU question on computer security, the model predicted the correct answer from threshold 10.0 to 3.0 with logit probability over 24.0%. When threshold falls below 3.0, the split approach splits the token "security" (in "computer security") into "secur/it/y" and the model makes a wrong prediction. This suggests the impact of oversplitting and how it can dilute key information across subtokens and obscures the contextual information. This observation also calls into question whether next-token prediction entropy serves as an optimal signal for re-tokenization criteria.

Furthermore, when the tokenizer splits more aggressively ($\tau = 1.0$), we observe that the logit distribution across four multiple-choice options (A, B, C, D) collapses into a uniform distribution, suggesting that the model loses its semantic comprehension completely and is taking a random guess. This also explains the seemingly recovery at low threshold values since random guesses yield higher accuracy than answers misled by token sequences inconsistent with the token distribution during pretraining.

6.3 Korean Tokenizer Analysis

The results above evaluate dynamic tokenization on Mistral-7B, whose English-centric vocabulary represents Korean at the byte level. Before turning to other models, we analyze how a tokenizer's Korean training data shapes its base segmentation—and hence what room the merge and split stages have to operate.

6.3.1 Effect of Korean Training Data on Tokenization Efficiency

A model's ability to represent Korean text efficiently is largely determined by how much Korean data was present during tokenizer training. Table 3 compares token counts for the sentence *'심혈관 질환의 예방을 위해 규칙적인 운동이 필요합니다.'* (Prevention of cardiovascular disease requires regular exercise) across seven models.

Mistral-7B tokenizes the sentence into 36 tokens because its 32,000-entry vocabulary contains almost no Korean syllables; individual syllables fall back to 3 raw UTF-8 bytes each. Models trained on large Korean corpora (Polyglot-ko, ko-gpt-trinity) tokenize the same sentence into 13–15 tokens, approaching the natural word unit of Korean. Korean text is written as a sequence of *eojeols*—space-delimited orthographic units that typically combine a content morpheme with one or more grammatical particles—and tokenizing at this granularity preserves morphosyntactic boundaries that are linguistically meaningful. This difference in base tokenization efficiency is a key fac-

Table 3: Token counts for a 7-word Korean sentence across tokenizers. Models trained on Korean data tokenize more efficiently.

Model	Korean Training Data	# Tokens	Tokenization Style
Mistral-7B	Not disclosed (English-dominant)	36	Byte-level fallback
Llama-3-KoEn-8B	∼80B tokens	20	Mixed syllable/byte
Qwen2.5-7B	18T multilingual tokens	19	Byte-merged syllables
Llama-2-ko-7B	∼40B KO tokens	17	Syllable-level
Polyglot-ko-1.3B	213B KO tokens (dedicated)	15	Word-level
Polyglot-ko-5.8B	213B KO tokens (dedicated)	15	Word-level
ko-gpt-trinity	35B KO tokens (web + news + wiki)	13	Eojeol-level

tor in understanding why dynamic merge operations interact differently with Korean-trained versus English-dominant models.

6.3.2 TOKEN-LEVEL VISUALIZATION

Table 4 shows how each tokenizer segments the sentence *"심혈관 질환의 예방을 위해 규칙적인 운동이 필요합니다."* Models with more Korean training data produce coarser, more linguistically meaningful tokens, while English-dominant models fall back to byte- or syllable-level fragmentation.

Table 4: Tokenization examples of across all seven models.

Model	n	Tokens
Mistral-7B	36	[심] [<ED>][<98>][<88>] [관] [질] [환] [의] [예] [방] [을] [위] [해] [규] [<bytes>] [적] [인] [운] [동] [이] . . .
Llama-3-KoEn-8B	20	[심] [<bytes>] [관] [질] [환] [의] [예방] [을] [위해] [규칙] [적] [인] [운동] [이] [필요] [합니다] [.]
Qwen2.5-7B	19	[심] [혈] [관] [질] [환] [의] [예] [방] [을] [위해] [규] [칙] [적인] [운] [동] [이] [필요] [합니다] [.]
Llama-2-ko-7B	17	[심] [혈] [관] [질환] [의] [예방] [을] [위해] [규] [칙] [적인] [운동] [이] [필요] [합니다] [.]
Polyglot-ko-1.3B	15	[심] [혈관] [질환] [의] [예방] [을] [위해] [규칙] [적] [인] [운동] [이] [필요] [합니다] [.]
Polyglot-ko-5.8B	15	[심] [혈관] [질환] [의] [예방] [을] [위해] [규칙] [적] [인] [운동] [이] [필요] [합니다] [.]
ko-gpt-trinity	13	[심] [혈] [관] [질환] [의] [예] [방을] [위해] [규칙] [적인] [운동이] [필요] [합니다.]

The disparity in tokenization granularity raises a question about the validity of evaluating entropy-guided splitting solely on Mistral-7B: if the base tokenizer already operates at the byte level for Korean, the split stage has no coarser units to decompose. To assess whether entropy-guided splitting produces qualitatively different outcomes when applied to models with richer Korean vocabularies, we extend our evaluation to models trained on substantial Korean corpora.

6.4 MULTI-MODEL RESULTS ON KOBEST AND KMMLU

To examine whether the findings on Mistral-7B generalize, we evaluate six additional models on KoBEST (full test splits, all 5 tasks) and KMMLU (all 45 subjects) under `plain` and `entropy_split` ($\tau = 3.0$) conditions. The models span a range of Korean training data richness: Korean-dedicated corpora (Polyglot-ko-1.3B, Polyglot-ko-5.8B, ko-gpt-trinity-1.2B), Korean-extended architectures (Llama-2-ko-7B, Llama-3-KoEn-8B), and large multilingual models (Qwen2.5-7B).

Table 5 shows KoBEST accuracy across all models and tasks.

Table 5: KoBEST accuracy (%) for plain and entropy-split tokenization ($\tau = 3.0$) across seven models. P = plain, S = entropy_split.

Model	COPA		BoolQ		WiC		HellaSwag		SentiNeg	
	P	S	P	S	P	S	P	S	P	S
Mistral-7B	**57.5**	56.9	15.5	**47.9**	42.2	**48.8**	52.2	**52.4**	93.7	**95.0**
Polyglot-ko-1.3B	**71.6**	67.3	45.2	**50.2**	**48.8**	48.7	**49.2**	42.2	**96.5**	55.2
Polyglot-ko-5.8B	**75.0**	70.4	42.2	**46.7**	**48.9**	48.7	**53.0**	49.6	**95.5**	50.9
Llama-2-ko-7B	**65.0**	63.4	27.1	**49.4**	46.9	**48.8**	**48.2**	47.6	**95.7**	66.5
Llama-3-KoEn-8B	**74.6**	68.7	13.8	**45.7**	48.4	**49.0**	**60.6**	53.6	**99.2**	91.4
Qwen2.5-7B	63.9	**65.6**	8.5	**20.2**	30.8	**46.0**	**56.2**	53.2	**97.0**	96.2

Two consistent patterns emerge across models. First, SentiNeg accuracy degrades substantially under entropy_split for Korean-trained base models (e.g., Polyglot-ko-1.3B: 96.5% $\rightarrow$ 55.2%; Polyglot-ko-5.8B: 95.5% $\rightarrow$ 50.9%), while instruction-tuned models are largely unaffected (Mistral-7B: 93.7% $\rightarrow$ 95.0%; Qwen2.5-7B: 97.0% $\rightarrow$ 96.2%). We attribute this to the nature of the task: base models may rely on surface-level lexical cues that are disrupted when token boundaries shift, whereas instruction-tuned models appear to have more robust semantic representations.

Second, BoolQ accuracy improves under entropy_split across nearly all models, in some cases substantially (Mistral-7B: 15.5% $\rightarrow$ 47.9%; Llama-3-KoEn: 13.8% $\rightarrow$ 45.7%), suggesting that finer-grained token boundaries facilitate alignment between a passage and a yes/no question. For COPA, WiC, and HellaSwag, effects are modest and inconsistent in direction, indicating that tasks requiring broader contextual understanding are less sensitive to token boundary changes. Overall, entropy_split does not uniformly improve or harm performance; its effect depends on task type and model training regime.

7 Conclusion

We presented a two-stage merge–split dynamic tokenization pipeline and used it to dissect when inference-time changes to token boundaries help. Across English and Korean benchmarks and seven models, no merge-only, split-only, or merge–split configuration surpassed the frozen baseline: moving the token distribution away from what the model saw in pretraining consistently costs accuracy, and predicting embeddings for novel merged tokens with a hypernetwork is the dominant source of degradation—severely so for Korean, whose syllable- and byte-level segmentation under an English-centric tokenizer leaves little structure for either stage to exploit. Splitting is best viewed as a corrective on over-merging: it recovers much of the accuracy lost to aggressive cross-word merging but does not raise raw performance, and our controls indicate that next-token entropy is only a weakly informative split signal. These findings temper the promise of retrofitted dynamic tokenization and suggest two directions: better embedding prediction for out-of-distribution merged tokens, and richer split criteria than next-token entropy.

Author Contributions

Justin Chae contributed to implementing and running evaluations of each method on MMLU and KMMLU. **Hanbyul Kang** contributed to the literature review on Korean NLP, designing a Korean-specific character-level split method, and conducting Korean benchmark evaluations and multi-model analysis.
Alvin Zeng contributed to designing the entropy-driven split strategy, implementing varying threshold experiment and conducting case studies on the MMLU benchmark results.

References

Marco Cognetta, Ana Kobzeva, Nguyen Ngo, Taro Todo, Nam Bui, and Taro Watanabe. Tokenization is more than compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

Darius Feher, Ivan Vulić, and Benjamin Minixhofer. Retrofitting large language models with dynamic tokenization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 29866–29883, 2025.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

Myeongjun Jang, Deuk-Sin Kim, Dong-Shin Kwon, and Ernest Davis. KoBEST: Korean balanced evaluation of significant tasks. In *Proceedings of the 29th International Conference on Computational Linguistics*, 2022.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018.

Alicia Liu et al. SuperBPE: Space travel for language models. *arXiv preprint arXiv:2503.13423*, 2025.

Benjamin Minixhofer, Edoardo M Ponti, and Ivan Vulić. Zero-shot tokenizer transfer. *Advances in Neural Information Processing Systems*, 37:46791–46818, 2024.

Kyubyong Park, Joohong Lee, Seongbo Jang, and Dawoon Jung. An empirical study of tokenization strategies for various Korean NLP tasks. In *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, 2020.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness across languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3118–3135, 2021.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.

Guijin Son, Hanwool Lee, Sungdong Kim, Seungone Kim, et al. KMMLU: Measuring massive multitask language understanding in Korean. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

A Appendix

A.1 MMLU accuracies over varying threshold values

Table 6: Select MMLU Subject Accuracies Across Varying Entropy Thresholds (τ).

Subject	$\tau = 1$	$\tau = 2$	$\tau = 3$	$\tau = 4$	$\tau = 7$	$\tau = 10$
business ethics	0.40	0.50	0.50	0.51	0.56	0.58
college biology	0.47	0.51	0.55	0.53	0.56	0.69
college computer science	0.38	0.35	0.36	0.36	0.40	0.54
moral scenarios	0.24	0.23	0.23	0.24	0.24	0.24
us foreign policy	0.58	0.65	0.72	0.75	0.81	0.85
Overall accuracy	**0.43**	**0.47**	**0.49**	**0.51**	**0.54**	**0.60**